

**Отчет по выполнению
Индивидуального Домашнего Задания
Вариант 35**

Сделано
Жарковой Татьяной Владимировной
студенткой 2 курса Программной инженерии
группы БПИ-253

Москва 2026
Высшая школа экономики

Оглавление

Аннотация	3
Формулировка задания	4
Основные структуры данных	6
Конфигурация	9
Очередь событий	10
Основной цикл симуляции	11
Обработка поступления вызова	11
Выбор бригады	12
Стратегии выбора бригады	12
Диспетчерство	12
Прибытие бригады	13
Завершение работы	13
Возвращение бригады на базу	14
Повышение приоритета	14
Отмена вызова	15
Необслуженные вызовы	15
Отображение текущего состояния	15
Итоговая статистика	16
Логирование	17
Задержка визуализации	18
Обработка аргументов командной строки	18
Обработка прерывания программы	19
Использование стандартной библиотеки и системных средств	19
Проверка корректности программы	19

Аннотация

В данном отчете представлена работа индивидуального домашнего задания, вариант 35. Работа выполнена на языке С в личном репозитории.

Программа представляет собой событийную симуляцию работы диспетчерской службы. Она моделирует поступление экстренных вызовов, поиск подходящей свободной бригады, выезд бригады к месту происшествия, выполнение работ, завершение обслуживания и возвращение бригады на базу.

Симуляция выполняется последовательно по времени. Все происходящие в системе действия представлены в виде событий, помещаемых в очередь. На каждом шаге выбирается событие с минимальным временем наступления, текущее модельное время устанавливается равным времени события, после чего выполняется соответствующий обработчик.

Формулировка задания

В диспетчерскую службу поступают вызовы из разных районов города. Каждый вызов имеет уникальный номер, координаты, категорию срочности и требуемую специализацию бригады. Бригады различаются специализацией, скоростью движения и текущим местоположением. Диспетчер назначает свободную подходящую бригаду. После назначения бригада едет к месту вызова, оказывает помощь и затем становится доступной в текущем районе либо возвращается на базу. Срочность вызова может повышаться по мере ожидания. Один вызов не должен быть одновременно назначен нескольким бригадам, если условием явно не требуется совместная работа.

Требуется разработать консольное приложение, моделирующее работу службы экстренной помощи.

Входные параметры

- карта районов и время перемещения;
- количество и специализации бригад;
- число, время и место поступления вызовов;
- категории срочности;
- длительность оказания помощи;
- правила повышения приоритета;
- стратегия назначения и возврата бригад;
- максимальное время ожидания.

Отображаемые события

Необходимо отображать поступление вызова, изменение срочности, поиск и назначение бригады, выезд, прибытие, начало и окончание помощи, освобождение бригады, отмену и вызов, оставшийся без обслуживания.

Инварианты

- одна бригада обслуживает не более одного вызова;
- назначенная бригада соответствует требуемой специализации;
- один обычный вызов назначается не более чем одной бригаде;
- завершённый или отменённый вызов не назначается повторно;
- состояние доступности бригады изменяется согласованно с её маршрутом и работой.

Завершение

Моделирование завершается после окончания поступления вызовов и обработки всех возможных заявок либо по завершении смены с фиксацией необслуженных вызовов, либо по прерыванию пользователем, если задан режим неограниченного по времени выполнения.

Диспетчер, каждая бригада и каждый вызов являются самостоятельными участниками модели.

Основные структуры данных

Основные сущности модели описаны в файле `model.h`.

1. Район

Для каждого района хранится его название и таблица времени перемещения между районами. Массив `travel` используется для определения расстояния между текущим районом бригады и районом вызова.

2. Бригада

Для каждой бригады хранятся:

- идентификатор;
- специализация;
- скорость;
- текущий район;
- базовый район;
- текущее состояние;
- идентификатор обслуживаемого вызова.

Состояние бригады может принимать следующие значения:

- TEAM_AVAILABLE
- TEAM_EN_ROUTE
- TEAM_AT_SCENE
- TEAM_RETURNING

Таким образом, в каждый момент моделирования можно определить, свободна ли бригада, находится ли она в пути, работает ли на месте происшествия или возвращается на базу.

3. Вызов

Для каждого вызова хранятся:

- идентификатор;
- район возникновения;
- координаты места происшествия;
- требуемая специализация;
- приоритет;
- время поступления;
- длительность обслуживания;
- максимально допустимое время ожидания;
- время до повышения приоритета;
- время до отмены;
- текущее состояние;
- идентификатор назначенной бригады;
- время назначения бригады;
- время начала обслуживания;
- время завершения обслуживания.

Состояние вызова может принимать следующие значения:

- CALL_PENDING
- CALL_WAITING
- CALL_DISPATCHED
- CALL_EN_ROUTE
- CALL_IN_SERVICE
- CALL_COMPLETED
- CALL_CANCELLED
- CALL_UNSERVED

В каждый момент моделирования можно определить, поступил ли вызов в систему, ожидает ли он назначения бригады, находится ли назначенная бригада в пути, выполняется ли обслуживание, завершён ли вызов, отменён он или остался необслуженным.

4. Событие

Для каждого события хранятся:

- время наступления события;
- тип события;
- идентификатор объекта, связанного с событием;
- порядковый номер события.

Тип события может принимать следующие значения:

- EV_CALL_ARRIVAL
- EV_ESCALATION
- EV_TEAM_ARRIVAL
- EV_SERVICE_DONE
- EV_RETURN_DONE
- EV_CANCEL

Таким образом, каждое событие однозначно определяет, что должно произойти, в какой момент моделирования и с каким вызовом или бригадой связано это действие. Порядковый номер используется для определения порядка обработки событий, если несколько событий имеют одинаковое время наступления.

5. Симуляция работы службы

Для симуляции хранятся:

- список районов;
- количество районов;
- список бригад;
- количество бригад;
- список вызовов;
- количество вызовов;
- очередь событий;
- количество событий в очереди;
- порядковый номер следующего события;
- текущее модельное время;

- задержка отображения событий;
- ограничение продолжительности смены;
- признак включения эскалации приоритетов;
- признак возврата бригад на базу;
- выбранная стратегия назначения бригад;
- суммарное время ожидания;
- суммарное время обслуживания;
- количество обслуженных вызовов;
- количество отменённых вызовов;
- количество необслуженных вызовов;
- количество повышений приоритета;
- количество назначений бригад;
- количество обработанных событий;
- файл журнала моделирования.

Структура содержит всё текущее состояние модели: объекты предметной области, очередь предстоящих событий, параметры моделирования, текущее модельное время и накопленную статистику. Благодаря этому на каждом шаге можно определить состояние всей системы и обработать следующее событие.

Конфигурация

Начальные параметры системы считываются из текстового конфигурационного файла.

В конфигурации задаются:

- задержка визуализации;
- продолжительность смены;
- включение механизма эскалации;
- необходимость возврата бригад на базу;
- стратегия выбора бригады;
- список районов;
- список бригад;
- список вызовов.

После чтения конфигурации выполняется проверка корректности данных. Проверяются диапазоны параметров, уникальность идентификаторов, существование районов и специализаций, корректность временных характеристик и другие ограничения модели.

```
test_single_call.txt X
emergency_dispatch > tests > test_single_call.txt
1  delay_ms=0
2  shift_limit=50
3  escalation_enabled=1
4  return_to_base=1
5  strategy=NEAREST
6
7  [DISTRICTS]
8  BASE 0 1
9  NORTH 1 0
10
11 [TEAMS]
12 1 MEDIC 30 BASE
13
14 [CALLS]
15 1 0 NORTH 1 1 1 MEDIC 5 20 0 0
```

Пример конфигурации теста single_call

Очередь событий

События обрабатываются в порядке очереди. При добавлении события оно само получает время наступления и порядковый номер. При извлечении выбирается ближайшее по времени событие.

После обработки события могут быть созданы новые события. Например, после назначения бригады создаётся событие её прибытия, а после начала обслуживания — событие окончания обслуживания.

Таким образом, заранее моделировать каждую минуту не требуется. Программа переходит непосредственно к моментам, в которых происходят изменения состояния системы.

Основной цикл симуляции

Общий алгоритм имеет следующий вид:

1. Вывести информацию о начале смены.
2. Создать начальные события поступления вызовов.
3. Извлечь ближайшее событие из очереди.
4. Установить текущее модельное время.
5. Проверить истечение допустимого времени ожидания вызовов.
6. Передать событие соответствующему обработчику.
7. При необходимости вывести текущее состояние системы.
8. Повторять действия, пока очередь событий не станет пустой или моделирование не будет прервано.

Обработка поступления вызова

При событии EV_CALL_ARRIVAL вызывается обработчик поступления вызова.

Вызов переводится из состояния CALL_PENDING в CALL_WAITING.

После этого:

1. информация о вызове выводится пользователю;
2. при необходимости планируется событие повышения приоритета;
3. при наличии ограничения на отмену планируется событие отмены;
4. выполняется попытка немедленно подобрать подходящую свободную бригаду.

Если подходящая бригада найдена, вызов сразу передаётся на обслуживание.

Выбор бригады

Отбрасываются бригады, которые:

- заняты;
- имеют неподходящую специализацию;
- не могут добраться до района вызова.

После этого выбор зависит от установленной стратегии.

Стратегии выбора бригады

В программе реализованы две стратегии:

- STRATEGY_NEAREST
- STRATEGY_FASTEST

Стратегия NEAREST

При стратегии NEAREST в первую очередь выбирается бригада, находящаяся на минимальном расстоянии от района вызова.

При одинаковом расстоянии дополнительно учитывается время прибытия, а затем идентификатор бригады.

Стратегия FASTEST

При стратегии FASTEST выбирается бригада с минимальным временем прибытия.

Время зависит от расстояния между районами и скорости конкретной бригады.

Диспетчерство

При назначении бригады учитываются:

- доступность бригады;

- соответствие специализации;
- выбранная стратегия;
- приоритет вызова;
- длительность ожидания при одинаковом приоритете

В первую очередь обслуживаются вызовы с более высоким приоритетом. Если приоритет одинаковый, преимущество получает вызов, который ожидает дольше.

После назначения:

CALL_WAITING становится CALL_EN_ROUTE

TEAM_AVAILABLE становится TEAM_EN_ROUTE

Прибытие бригады

Бригада перемещается в район вызова.

TEAM_EN_ROUTE становится TEAM_AT_SCENE

CALL_EN_ROUTE становится CALL_IN_SERVICE

После этого в очередь добавляется событие завершения работы

Завершение работы

Вызов выполняется.

CALL_IN_SERVICE становится CALL_COMPLETED

В статистику добавляется:

- время ожидания;
- продолжительность обслуживания;
- количество обслуженных вызовов.

Время ожидания вычисляется как разница между моментом начала обслуживания и временем поступления вызова.

После завершения обслуживания возможны два варианта.

Если включён возврат на базу:

TEAM_AT_SCENE становится TEAM_RETURNING

и создаётся событие EV RETURN DONE.

Если возврат на базу отключён:

TEAM_AT_SCENE становится TEAM_AVAILABLE

и бригада сразу может быть назначена на следующий вызов.

Возвращение бригады на базу

При событии EV_RETURN_DONE бригада перемещается на базу и становится готовой к работе.

TEAM_RETURNING становится TEAM_AVAILABLE

После этого снова выполняется попытка обслуживания вызовов.

Повышение приоритета

Для ожидающих вызовов предусмотрен механизм повышения приоритета.

Если вызов ожидает заданное количество времени, его приоритет повышается.

После повышения приоритета выполняется повторная попытка выполнения вызова.

Отмена вызова

Для некоторых вызовов может быть задано максимальное время ожидания до отмены.

Если вызов остаётся в состоянии CALL_WAITING и превышает допустимое время ожидания, то:

CALL_WAITING становится CALL_CANCELLED

Количество таких случаев учитывается в статистике.

Необслуженные вызовы

Если вызов ожидает слишком долго и достигает max_wait, то:

CALL_WAITING становится CALL_UNSERVED

Количество необслуженных вызовов учитывается отдельно от отменённых.

Отображение текущего состояния

Для контроля работы симуляции программа периодически выводит снимок текущего состояния системы.

Отображаются:

- количество ожидающих вызовов;
- количество событий в очереди;
- состояние каждой бригады;
- её текущий район;
- обслуживаемый вызов.

Такой вывод позволяет наблюдать за изменением состояния системы непосредственно во время выполнения программы.

LIVE STATUS				
Waiting calls: 0 Events: 4				
B1	MEDIC	AT_SCENE	district=WEST	call=#016
B2	MEDIC	AVAILABLE	district=NORTH	call=---
B3	REANIM	AVAILABLE	district=EAST	call=---
B4	REANIM	AVAILABLE	district=WEST	call=---
B5	FIRE	AT_SCENE	district=SOUTH	call=#017
B6	POLICE	AVAILABLE	district=BASE	call=---

Пример текущего состояния при config.txt

Итоговая статистика

После завершения моделирования вычисляются и выводятся основные показатели:

- количество поступивших вызовов;
- количество обслуженных вызовов;
- количество отменённых;
- количество необслуженных;
- количество диспетчеризаций;
- количество повышений приоритета;
- среднее время ожидания;
- среднее время обслуживания;
- момент окончания моделирования.

[00:12:00] SHIFT FINISHED

SHIFT SUMMARY	
Calls received:	18
Served:	18
Cancelled:	0
Unserved:	0
Dispatches:	18
Priority escalations:	0
Average waiting:	2.44 min/call
Average service:	13.06 min/call
Simulation end:	720 min

Пример статистики при config.txt

Логирование

Для сохранения истории работы программы используется файл логирования.

Функция логирования одновременно выводит сообщение в стандартный вывод и, если задан файл журнала, записывает его в файл.

Каждое событие получает временную метку:

После завершения смены в журнал также записывается итоговая статистика.

```
test_single_call.log X
emergency_dispatch > tests > test_single_call.log
1 [00:00:00] SHIFT STARTED: districts=2 teams=1 calls=1
2 [00:00:00] CALL □#001 ARRIVED: district=NORTH priority=1 spec=MEDIC
3 [00:00:00] DISPATCH: CALL □#001 priority=1 spec=MEDIC -> B1 (NEAREST), TIME=1 min
4 [00:00:01] B1 ARRIVED at NORTH for CALL □#001; service started
5 [00:00:06] CALL □#001 SERVICE COMPLETED by B1; total wait=1 min
6 [00:00:06] B1 RELEASED; returning to base BASE, TIME=1 min
7 [00:00:07] B1 RETURNED to base BASE and is AVAILABLE
8 [00:00:50] SHIFT FINISHED
9
10 === SHIFT SUMMARY ===
11 Calls received: 1
12 Served: 1
13 Cancelled: 0
14 Unserved: 0
15 Dispatches: 1
16 Priority escalations: 0
17 Average waiting: 1.00 min/call
18 Average service: 5.00 min/call
19 Simulation end: 50 min
```

Пример логирования теста single_call

Задержка визуализации

Параметр `delay_ms` используется только для визуального замедления работы программы. Меняется только скорость отображения происходящего пользователю.

Обработка аргументов командной строки

Программа поддерживает следующие параметры:

- -c config
- -d delay_ms
- -s nearest|fastest
- -l logfile
- -h

Параметры командной строки имеют приоритет над соответствующими значениями конфигурационного файла.

Обработка прерывания программы

Программа поддерживает нормальное завершение и завершение по сигналу SIGINT.

Основной цикл периодически проверяет значение флага. После получения Ctrl+C моделирование прекращается корректно.

Активные вызовы, которые не были завершены к моменту прерывания, переводятся в состояние CALL_UNSERVED, после чего формируется итоговая статистика.

Использование стандартной библиотеки и системных средств

В программе используются стандартные средства языка C и POSIX.

- `stdio.h` — работа с файлами и стандартным вводом/выводом;
- `stdlib.h` — стандартные функции;
- `string.h` и `strings.h` — обработка строк;
- `stdarg.h` — реализация функции форматированного журналирования с переменным количеством аргументов;
- `time.h` — функция `nanosleep`;
- `signal.h` — обработка SIGINT;
- `limits.h` — константы для поиска минимальных значений

Для работы с конфигурацией и журналом используются файловые потоки `FILE *`, а ввод и вывод выполняются посредством стандартных функций библиотеки C.

Проверка корректности программы

Для проверки корректности работы программы был подготовлен набор тестовых конфигураций. Каждая конфигурация предназначена для проверки отдельного сценария работы диспетчерской службы:

- `test_cancel.txt` — проверка отмены вызова, если время ожидания превышает заданное значение;

- test_escalation.txt — проверка повышения приоритета вызова при превышении времени ожидания;
- test_fastest.txt — проверка стратегии назначения FASTEST, при которой выбирается бригада с минимальным временем прибытия;
- test_nearest.txt — проверка стратегии назначения NEAREST, при которой выбирается ближайшая доступная бригада;
- test_no_team.txt — проверка ситуации, когда в системе отсутствует доступная бригада требуемой специализации;
- test_shift_limit.txt — проверка ограничения продолжительности смены;
- test_single_call.txt — проверка базового сценария с одним вызовом и одной бригадой

Например, для проверки выбора бригады по стратегии FASTEST использовалась конфигурация:

```

≡ test_fastest.txt ×
emergency_dispatch > tests > ≡ test_fastest.txt
1  delay_ms=0
2  shift_limit=50
3  escalation_enabled=0
4  return_to_base=0
5  strategy=FASTEST
6
7  [DISTRICTS]
8  BASE 0 10 2
9  NORTH 10 0 8
10 EAST 2 8 0
11
12 [TEAMS]
13 1 MEDIC 30 BASE
14 2 MEDIC 60 NORTH
15
16 [CALLS]
17 1 0 EAST 1 1 1 MEDIC 5 20 0 0

```

Логирование

```
test_fastest.log X
emergency_dispatch > tests > test_fastest.log
1 [00:00:00] SHIFT STARTED: districts=3 teams=2 calls=1
2 [00:00:00] CALL □#001 ARRIVED: district=EAST priority=1 spec=MEDIC
3 [00:00:00] DISPATCH: CALL □#001 priority=1 spec=MEDIC -> B1 (FASTEST), TIME=2 min
4 [00:00:02] B1 ARRIVED at EAST for CALL □#001; service started
5 [00:00:07] CALL □#001 SERVICE COMPLETED by B1; total wait=2 min
6 [00:00:07] B1 RELEASED and AVAILABLE in district EAST
7 [00:00:50] SHIFT FINISHED
8
9 === SHIFT SUMMARY ===
10 Calls received: 1
11 Served: 1
12 Cancelled: 0
13 Unserved: 0
14 Dispatches: 1
15 Priority escalations: 0
16 Average waiting: 2.00 min/call
17 Average service: 5.00 min/call
18 Simulation end: 50 min
19
```